Reviewer Pack — Minimal Order of Affine Hecke Algebras vs AC0 Circuit Depth

Entry 01d3fa5cc32f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 18:49:23 UTC

Minimal Order of Affine Hecke Algebras vs AC0 Circuit Depth

Entry ID: `01d3fa5cc32f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 18:49:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Affine Hecke Algebras) **Field B** (complexity object): Complexity Theory: AC0 Circuit Depth

Statement:

> ['For every CNF formula, the minimal order of an element in the affine Hecke algebra associated with the boolean cube is polynomially related to the depth of the smallest AC0 circuit for the formula.', 'Equivalently, if f is a CNF formula, then the minimal order of an element in the affine Hecke algebra A(f) is O(poly(n)), where n is the number of variables in f, and this order determines the depth of the smallest AC0 circuit computing f.']

Rationale (proposer's reasoning):

> ['Affine Hecke algebras provide a rich algebraic structure that can encode combinatorial properties. Their minimal orders have been studied in algebraic combinatorics but rarely applied to complexity theory. A connection here could reveal new insights into both fields.', 'AC0 circuit depth is a fundamental measure of computational efficiency. If there is a direct relationship between the minimal order of affine Hecke algebras and AC0 circuit depth, it suggests a deeper structure that may lead to efficient algorithms or circuits.']

Taxonomy category: `Affine_Hecke` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9d984c4561f3f33f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the minimal order of an element in the affine Hecke algebra is O(poly(n)), where n is the number of variables, and this order correlates with AC0 circuit depth such that at least 80% of the generated CNF formulas have a correlation coefficient greater than or equal to 0.8. The conjecture is falsified if any seed produces a minimal order greater than O(poly(n)) or a correlation coefficient less than 0.6.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal order Affine Hecke algebras AND AC0 circuit depth` - `CNF formula Affine Hecke algebra element AND circuit depth` - `Affine Hecke algebra minimal order AND related to AC0 circuit`

Top relevant hits considered: - [http://arxiv.org/abs/1405.6705v5] A categorical equivalence between affine Yokonuma-Hecke algebras and some quiver Hecke algebras - [http://arxiv.org/abs/1310.6668v2] Dirac cohomology for the degenerate affine Hecke Clifford algebra - [http://arxiv.org/abs/1311.3745v2] Affine Hecke algebras and quiver Hecke algebras - [http://arxiv.org/abs/q-alg/9710037v2] Lie algebras and degenerate Affine Hecke Algebras of type A

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for j in range(i)):
                clauses.append(clause)
        return clauses

    def min_order_affine_hecke(cnf):
        n = len(cnf[0])
        order = 2**n
        return order

    def ac0_circuit_depth(cnf):
        n = len(cnf[0])
        depth = n + 1
        return depth
```

```
    cnf = generate_cnf(40)
    min_order = min_order_affine_hecke(cnf)
    circuit_depth = ac0_circuit_depth(cnf)

    if min_order > 2**(n**2):
        return {
            "metric_name": "Minimal Order vs AC0 Circuit Depth",
            "metric_value": min_order,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "min_order_greater_than_poly_n"
        }

    correlation_coefficient = 1.0
    if circuit_depth > 0:
        correlation_coefficient = min_order / circuit_depth

    return {
        "metric_name": "Minimal Order vs AC0 Circuit Depth",
        "metric_value": correlation_coefficient,
        "instances_tested": 1,
        "conjecture_holds": correlation_coefficient >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='min_order_greater_than_poly_n' first_failing_see
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cb1f77c.py", line 73, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cb1f77c.py", line 39, in run_trial
    cnf = generate_cnf(40)
          ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cb1f77c.py", line 25, in generate_cnf
    if all(clause[i] != -clause[j] for j in range(i)):
                                                  ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot verify if the conjecture's conditions are met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11393
2	preregistration	ollama_remote	glm4:latest	0	0	5974
3	novelty	ollama_remote	glm4:latest	0	0	4620
4	novelty	ollama_remote	glm4:latest	0	0	6248
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18687
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21427
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7473
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10339
9	judge	ollama_remote	glm4:latest	0	0	141157

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 227318 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/01d3fa5cc32f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/01d3fa5cc32f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/01d3fa5cc32f.tar.gz` (if generated)